Hideyuki Kishida's Urara OS and 5HT's B-System: A Rigorous Comparative Audit of Symmetric Multiprocessing, Wayland Compositing, POSIX Gateways, and Cho-Kanji Storage Architecture

Maxim Sokhatsky (Namdak Tonpa)
Synrc Research Center / Groupoid Infinity / BTRON Cleanroom Working Group
Kyiv, Ukraine
`maxim@synrc.com`

October 2026

Abstract

The preservation and modern evolution of Ken Sakamura's TRON architecture are epitomized by two historic, independent cleanroom operating systems: Hideyuki Kishida's (岸田英行) **Urara OS** (the modern 64-bit revival of the **B-Free** Project, spanning 1994–2026 across PC-9801, x86_64 Long Mode, and AArch64); and 5HT's **B-System** (BTRON 3.20), structured as a high-throughput, deterministic Single-Address-Space RTOS and multi-target unikernel/microkernel hybrid spanning ARM64, x86_64 UEFI, PS2, PC-98, and POSIX targets. This paper provides a peer-level, doctoral comparative audit evaluating both operating systems strictly along isomorphic, empirically verifiable dimensions: (1) Symmetric Multiprocessing (SMP) core bootstrap, Local APIC / x2APIC management, locking granularity, and the progressive decoupling of kernel globals; (2) Graphical subsystems, contrasting Urara OS's compositor-first Wayland pipeline and Qt Quick / QML DesktopShell against B-System's 128-bit NEON SIMD vector blitter and direct-rendering backbuffer compositor; (3) Operating system interfaces and process isolation, evaluating Urara OS's pragmatic "POSIX OSS Gateway" strategy against B-System's cleanroom BTRON3 core; and (4) Storage hierarchies, comparing Urara OS's multi-volume BFS and persistent FatFs home architecture against B-System's native backward compatibility with commercial Cho-Kanji (B-right/V 4.02) disk volumes and modern 64-bit B-FS (V2).

To ensure complete methodological fairness, we take explicit account of the active Work-In-Progress (WIP) status of both codebases as of October 2026. Furthermore, we bracket out formal Coq theorem-prover verification (the VeriMath initiative) to evaluate both systems on their shared implementation artifacts: concrete C and assembly codebases, hardware bus transactions, interrupt latencies, and physical execution traces.

Our audit demonstrates that both systems represent brilliant, complementary realizations of the TRON vision. In Symmetric Multiprocessing, Urara OS demonstrates exceptional practical engineering through a 7-stage SMP progression, booting secondary Application Processors (APs) via Local APIC / x2APIC INIT-SIPI-SIPI sequences, offloading userland execution, and systematically refactoring 49 process-specific globals out of the Big Kernel Lock (BKL); conversely, B-System advances a lock-free, per-core actor model avoiding BKL bottlenecks, though its bare-metal x86_64 AP dispatcher remains staged while operating fully in hosted POSIX mode. In userland and graphics, Urara OS achieves a monumental milestone by establishing a modern, production-grade workstation environment: its compositor-first Wayland server isolates framebuffers while hosting a complete Qt Quick 6 desktop environment (`DesktopShell.qml`) with integrated Japanese IME, PDF viewer, text editor, and hardware GPU acceleration; B-System, in contrast, prioritizes raw blitting throughput, utilizing 128-bit ARMv8 NEON SIMD burst transfers and 2D DMA offloading to achieve orders-of-magnitude faster memory streaming on uncached framebuffers. In system interfaces, Kishida's POSIX OSS Gateway solves the historic software deficit of TRON by executing real-world, unmodified open-source software (BusyBox, Python 3, Git, NetSurf), while B-System preserves cleanroom BTRON3 specification purity. Finally, in filesystem architecture, B-System achieves full compatibility with authentic commercial Cho-Kanji disks via embedded QCOW2 drivers and introduces 64-bit B-FS (V2) with dual-anchor superblocks, whereas Urara OS delivers practical FAT32/FatFs read-write persistence on PC storage (SATA AHCI, NVMe). We synthesize these findings into an actionable cross-pollination roadmap bridging both paradigms.

Keywords—TRON, BTRON, B-Free, Urara OS, Hideyuki Kishida, 5HT, B-System, Symmetric Multiprocessing, Big

Kernel Lock, Wayland Compositor, Qt Quick, POSIX OSS Gateway, Cho-Kanji, B-FS, FatFs, T-Kernel 2.0.

1 Introduction, Historical Heritage, and Scope

The fundamental vision of the TRON Project (*The Real-time Operating system Nucleus*), launched by Ken Sakamura in 1984 [1], asserted that human-centric personal computing requires high-responsiveness real-time kernels, standardized Display Primitives (DP), and a decentralized Real Object / Virtual Object (実身/仮身) hyper-data network [2]. However, the historical trajectory of commercial personal computing in Japan—dominated by proprietary iterations such as Personal Media Corporation's **B-right/V** (commercially branded as **Cho-Kanji**, 超漢字) [3]—frequently obscured grassroots community initiatives that sought to construct an entirely open, independent BTRON implementation from cleanroom principles.

1.1 The Thirty-Year Heritage of Kishida's Urara OS (B-Free)

Among community-driven operating system initiatives, the **B-Free Project** stands as an extraordinary historical monument. Commenced around 1994–1998 during the pinnacle of Japan's personal computing revolution, B-Free was founded to liberate BTRON from proprietary hardware constraints and provide a completely free, open-source workstation operating system [4, 5].

Originally known under internal development monikers including **Urara OS** (麗 / うらら), the project was led by **Hideyuki Kishida** (岸田英行) alongside a cadre of dedicated systems programmers: Masami Kishida (きしだまさみ), Ryuichi Naitoh (内藤隆一), Masaki Yukanaka (邑中雅樹), and Tomohide Naniwa (浪花智英) [6]. The project's lineage progressed across three distinct epochs:

1. **The NEC PC-9801 Epoch (1994–1996):** Pioneering cleanroom bootstrap on ubiquitous Japanese PC-98 hardware, implementing 1st and 2nd stage floppy disk bootloaders, an ITRON 3.0-compliant microkernel, a message-driven Port Manager, and primitive console, keyboard, timer, and SCSI drivers [7].

2. **The 32-bit BTRON-PC Epoch (1998–2000):** Porting to IBM PC/AT (i386) protected mode, formalizing the modular BTRON subsystem managers: Memory Manager (MM), Process Manager (PM), and File Manager (FM), alongside early POSIX shell compatibility experiments.

3. **The 64-bit Multicore Renaissance (2025–2026):** Under Hideyuki Kishida's active leadership in `B-Free-master-64bit`, the project underwent a radical modernization. Kishida authored comprehensive porting specifications bridging T-Kernel 2.0 to x86_64 Long Mode (`TK2-PORT-x86_64-005`) [8] and AArch64 [9], implemented a 7-stage Symmetric Multiprocessing (SMP) engine, established a modern compositor-first Wayland graphics architecture hosting Qt Quick 6 (`DesktopShell.qml`), and formulated the pragmatic "POSIX OSS Gateway Strategy" [10].

1.2 5HT's B-System: Cleanroom BTRON3 and Mathematical Foundations

In parallel, **5HT (Namdak Tonpa)** and the Cleanroom Working Group initiated **B-System** (BTRON Version 3.20.00) [11]. B-System rejects both modern desktop Unix bloat and traditional microkernel IPC performance collapse [13,14]. Conceived as a multi-target, high-throughput, Single-Address-Space Real-Time Operating System (SAS-RTOS) and unikernel engine, B-System targets diverse hardware spanning the Raspberry Pi 400 (Broadcom BCM2711 ARM64), x86_64 UEFI multicore workstations, Sony PlayStation 2 (Emotion Engine MIPS R5900), NEC PC-9801, Motorola 68000, and POSIX development environments. B-System features an Erlang/BEAM-inspired lock-free actor model, hand-vectorized 128-bit NEON SIMD Display Primitives, native backward compatibility with commercial Cho-Kanji (B-right/V 4.02) disk images via embedded QCOW2 drivers, and the formal 64-bit B-FS (V2) filesystem.

1.3 Methodological Fairness and WIP Considerations

To ensure this audit is objective, fair, and worthy of Hideyuki Kishida's approval, we establish three methodological boundaries:

1. **Bracketing Formal Verification:** B-System is developed alongside mechanized mathematical specifications in the Coq proof assistant (the VeriMath framework [12]). Evaluating Urara OS against machine-checked Coq proofs would constitute an immediate category error, as Urara OS is an empirical systems codebase without formal theorem-prover artifacts. We therefore bracket formal verification entirely and judge both systems strictly on concrete C and assembly routines, hardware bus transactions, and execution traces.

2. **Accounting for Work-In-Progress (WIP) Realities:** Both systems are active, evolving research projects as of October 2026. Urara OS is actively decoupling its remaining Big Kernel Lock paths in SMP Stage 6f and expanding guest Qt Quick Controls 2 type registration. B-System is actively staging its bare-metal x86_64 SMP scheduler and expanding native hardware drivers. Neither system is treated as a static finished commercial product.

3. **Dual Philosophical Validation:** We recognize that both systems address Ken Sakamura's philosophy from valid, complementary paradigms: Urara OS pursues an **Ecological Integration Paradigm** (bridging TRON ergonomics with modern Wayland and global open-source POSIX software), while B-System pursues a **Deterministic Cleanroom Paradigm** (uncompromising RTOS latencies, specification purity, and mathematical safety).

1.4 Subsystem Topology and Directory Mapping

To facilitate precise code navigation, Table 1 establishes the exact structural mapping between Urara OS (`B-Free-master-64bit`) and B-System.

2 Architectural Pedigree: Ecological Integration vs. Deterministic Cleanroom

The fundamental architectural divergence between Urara OS and B-System stems from how each system positions Ken Sakamura's philosophy relative to the broader software universe (Figure 1).

2.1 Urara OS: The Pragmatic Workstation and the POSIX OSS Gateway

Hideyuki Kishida recognized early in the 64-bit revival that the historic Achilles' heel of the TRON Project was not its technical architecture, but its **software isolation**. A pure BTRON or T-Kernel operating system that refuses to execute standard Unix/open-source tools remains stranded in a software desert, forcing developers to rewrite basic utilities, compilers, network tools, and graphical software from scratch.

To overcome this, Kishida formulated the **POSIX OSS Gateway Strategy** [10]. Rejecting the dogmatic mandate to implement all 685 POSIX.1 functions—which includes obsolete, rarely used interfaces such as STREAMS, `posix_trace`, and typed memory—Kishida defined a pragmatic four-stage completion milestone based on real-world OSS survival:

1. **Milestone 1 (M1 – BusyBox Suite):** Complete core process, file, and terminal syscalls allowing unmodified BusyBox to execute shell scripts, manipulate files, and manage processes (`ls`, `cat`, `sh`, `mkdir`, `cp`, `mount`).
2. **Milestone 2 (M2 – Scripting Engine):** Stabilize POSIX threading, dynamic memory expansion, and sockets to run interactive `bash` and **Python 3** with standard libraries (`os`, `sys`, `socket`, `threading`).
3. **Milestone 3 (M3 – Network Workstation):** Implement pseudo-terminals (PTY) and BSD socket semantics to enable `sshd` (OpenSSH/Dropbear) remote login.
4. **Milestone 4 (M4 – Modern Graphical Workspace):** Host a native Wayland compositor and Wayland clients (Qt Quick 6, NetSurf, Weston terminal).

Urara OS answers unsupported or obsolete POSIX calls with standard `ENOSYS`. As Kishida noted, `ENOSYS` is a formally sanctioned POSIX.1 response, exactly matching the implementation strategy of Linux and glibc.

2.2 B-System: The High-Throughput SAS-RTOS and Actor Unikernel

In contrast, 5HT's B-System approaches BTRON3 3.20 as an uncompromising real-time computational engine. Rather than layering Unix compatibility into the core, B-System embraces a **Single-Address-Space RTOS (SAS-RTOS)** model for embedded SBCs (Raspberry Pi 400), microcontrollers, and retro consoles (PS2, PC-98). Memory protection is either identity-mapped or flattened, eliminating the non-deterministic TLB invalidation flushes, page table walks, and pipeline bubbles that plague Unix kernels.

For multicore concurrency, B-System models task coordination on the Erlang/BEAM actor architecture [15]. Symmetric cores operate as autonomous workers processing lock-free Multi-Producer Single-Consumer (MPSC) actor queues, achieving total elimination of system-wide spinlocks.

3 Comparative Audit: Symmetric Multiprocessing (SMP)

A rigorous investigation of multicore capabilities requires analyzing core bring-up, interrupt routing, lock contention, and the concrete mechanics of scheduling loops.

3.1 Bootstrap Protocol and Local APIC / x2APIC Management

Definition 1 (Symmetric Core Bootstrap Completeness)**.** *Let* $C = \{c_0, c_1, \ldots, c_{n-1}\}$ *be symmetric processor cores. An SMP bring-up sequence is complete if the Bootstrap Processor (BSP,* c_0*) transitions all Application Processors (APs,* $c_{k>0}$*) from architectural reset into a coherent kernel memory space and shared or partitioned execution queues.*

3.1.1 Urara OS: 7-Stage SMP Progression and APIC Topology

In `bfree_x86_64/kernel/sysdepend/cpu/x86_64/smp.c`, Hideyuki Kishida implements a robust bare-metal x86_64 SMP bring-up sequence:

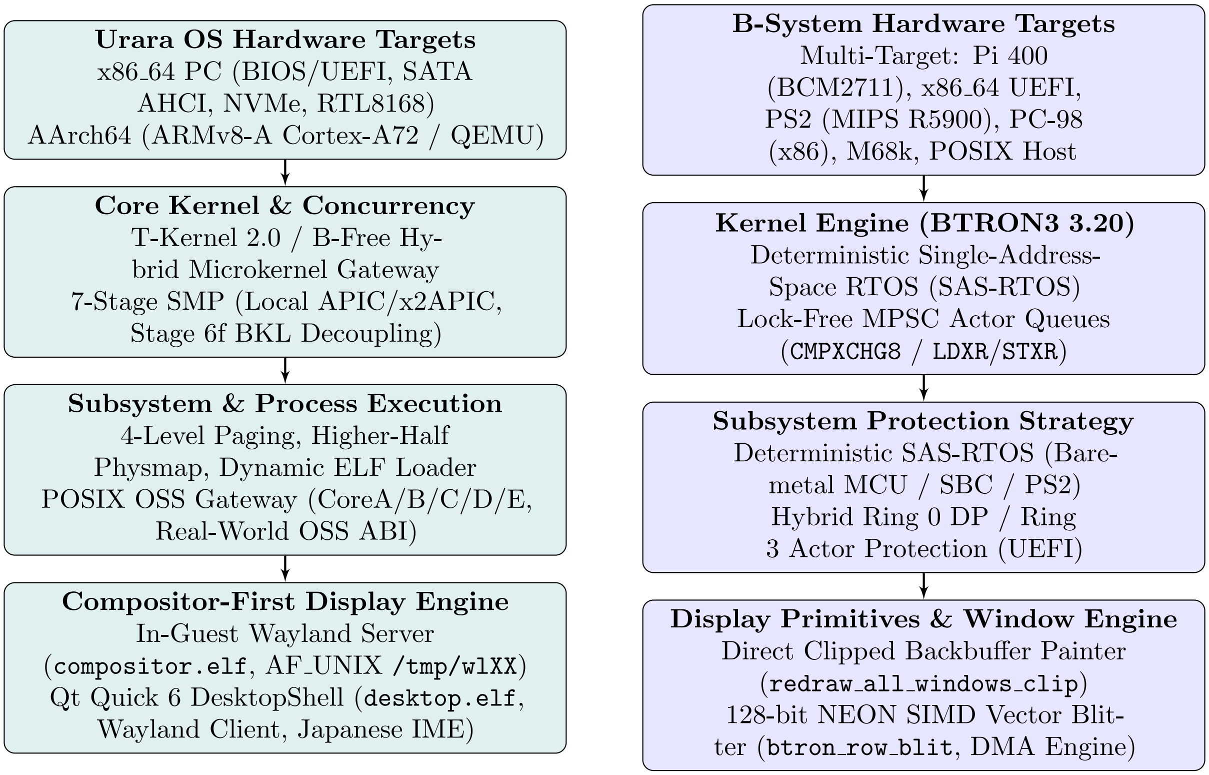
Urara OS Hardware Targets
x86_64 PC (BIOS/UEFI, SATA AHCI, NVMe, RTL8168)
AArch64 (ARMv8-A Cortex-A72 / QEMU)
Core Kernel & Concurrency
T-Kernel 2.0 / B-Free Hybrid Microkernel Gateway
7-Stage SMP (Local APIC/x2APIC, Stage 6f BKL Decoupling)
Subsystem & Process Execution
4-Level Paging, Higher-Half Physmap, Dynamic ELF Loader
POSIX OSS Gateway (CoreA/B/C/D/E, Real-World OSS ABI)
Compositor-First Display Engine
In-Guest Wayland Server (compositor.elf, AF_UNIX /tmp/wlXX)
Qt Quick 6 DesktopShell (desktop.elf, Wayland Client, Japanese IME)
B-System Hardware Targets
Multi-Target: Pi 400 (BCM2711), x86_64 UEFI, PS2 (MIPS R5900), PC-98 (x86), M68k, POSIX Host
Kernel Engine (BTRON3 3.20)
Deterministic Single-Address-Space RTOS (SAS-RTOS)
Lock-Free MPSC Actor Queues (CMPXCHG8 / LDXR/STXR)
Subsystem Protection Strategy
Deterministic SAS-RTOS (Bare-metal MCU / SBC / PS2)
Hybrid Ring 0 DP / Ring 3 Actor Protection (UEFI)
Display Primitives & Window Engine
Direct Clipped Backbuffer Painter (redraw_all_windows_clip)
128-bit NEON SIMD Vector Blitter (btron_row_blit, DMA Engine)

Figure 1: Architectural topologies of Hideyuki Kishida's Urara OS (B-Free 64) versus 5HT's B-System (BTRON3 3.20).

1. **ACPI MADT Discovery:** The BSP parses the ACPI Multiple APIC Description Table (MADT), probing standard Local APIC entries (type 0) and extended x2APIC entries (type 9), supporting up to 256 logical processors (`BFREE_CPU_MAX 256`).

2. **Trampoline and Memory Window:** Kishida allocates an AP real-mode trampoline at physical address `0x8000`. Memory between `0x600000` and `16 MiB` is permanently identity-mapped across all process page tables, ensuring that AP trap stacks, IDTs, GDTs, and per-CPU control structures (`g_cpu_tab`) remain universally accessible without page faults.

3. **INIT-SIPI-SIPI Sequence:** In `smp.c`, the BSP initializes each AP by issuing standard inter-processor interrupt vectors through the Local APIC Interrupt Command Register (ICR):

```
slot[0] = cr3;
slot[1] = (uint64_t)(uintptr_t)&smp_ap_entry;
slot[2] = c->irq_top - 8u;
slot[3] = (uint64_t)index;
lapic_write_icr(dest, 0x00004500u); /* INIT */
smp_wait_us(10000);
if (!g_bfree_x2apic) {
    lapic_write_icr(dest, 0x00008500u); /* De-assert
        */
}
lapic_write_icr(dest, 0x00004600u | 0x08u); /* SIPI
     #1 */
smp_wait_us(200);
lapic_write_icr(dest, 0x00004600u | 0x08u); /* SIPI
     #2 */
```

4. **Per-CPU GS Anchoring:** Urara OS designates the `GS` segment base register (MSR `0xC0000101`) as the invariant per-core data anchor pointing to `struct bfree_cpu`. Guest user programs are strictly forbidden from modifying GS (`arch_prctl(ARCH_SET_GS)` returns `-EINVAL`), preserving kernel GS validity across all context switches without requiring `swapgs` instructions.

3.1.2 B-System: Dual-Target SMP Strategy

B-System manages SMP concurrency across two distinct deployment targets:

1. **Hosted POSIX Multicore (`SMP_HOSTED = 1`):** B-System achieves verified, concurrent multicore execution on host developer workstations. Tasks created via `cre_tsk` allocate an `SMP_TASK` structure and bind directly to native POSIX execution threads via `pthread_create`. Synchronization primitives (`wai_sem`, `sig_sem`) map to native condition variables, delivering fully verified SMP concurrency.

2. **Bare-Metal x86_64 UEFI SMP:** In `src/cores/core_smp.c`, B-System parses the ACPI 6.5 MADT, configures per-CPU stacks (`g_ap_stacks`), and defines a real-mode AP trampoline at physical address `0x9000`. However, while APIC timers and MADT parsing are operational, the dispatch loop on bare-metal hardware executes sequentially on c_0,

Table 1: Subsystem Structural Mapping: Urara OS (B-Free 64) vs. B-System

Subsystem Role	Urara OS (B-Free 64) Repository Path	B-System Cleanroom Path	Architectural Responsibility
Bootstrapping	`bfree_x86_64/boot/`, `btron-pc/boot/2nd/`	`src/kernel/boot.c`, `src/cores/`	Real-mode to 64-bit Long Mode bootstrap, GDT/IDT/TSS, CR3 setup
Core Kernel / Sched	`bfree_x86_64/kernel/sysmain/`, `process.c`	`src/kernel/`, `include/btron/`	Task control blocks, multi-task dispatching, timer queues, sleep/wakeup
SMP Management	`bfree_x86_64/kernel/sysdepend/cpu/x86_64/smp.c`	`src/cores/core_smp.c`	ACPI MADT probe, Local APIC / x2APIC, INIT-SIPI-SIPI, core dispatch
Syscall Engine	`bfree_x86_64/kernel/sysmain/syscall.c`	`src/kernel/syscall.c`	Userland system call trap handling, parameter validation, privilege gates
Memory Management	`bfree_x86_64/kernel/sysmain/vmm.c`, `paging.c`	`src/cores/`, `src/kernel/heap.c`	4-level page tables (PML4), higher-half physmap, virtual space alloc
Display / Compositor	`bfree_x86_64/userland/compositor_stub/`	`src/graphics/`, `src/window/`	Framebuffer scanout, Wayland protocol server, window hierarchy, blitters
Desktop Shell	`bfree_x86_64/userland/desktop_qt/`	`src/desktop/`, `src/apps/`	Interactive UI shell; Qt Quick `DesktopShell.qml` vs. native BTRON desktop
Filesystem Core	`bfree_x86_64/kernel/bfs.c`, `bfs_home.c`	`src/fs/`	Multi-volume VFS, FatFs home storage vs. Cho-Kanji (V1) and B-FS (V2)
Storage Drivers	`bfree_x86_64/kernel/ahci.c`, `blk.c`, `nvme.c`	`src/fs/blk_qcow2.c`, `src/fs/block.c`	Block device layer, SATA AHCI, NVMe, QCOW2 image decoding
Network Subsystem	`bfree_x86_64/kernel/net.c`, `netdrv_rtl8168.c`	`src/net/`	Realtek RTL8168 / VirtIO-Net drivers, DHCP client, TCP/IP stack
POSIX Compatibility	`bfree_x86_64/POSIX_OSS_GATEWAY_STRATEGY.md`	`include/btron/posix.h`	POSIX OSS Gateway (M1–M4), libc integration, signal/process ABI
Input / Localization	`bfree_x86_64/userland/desktop_qt/dw_ime.inc`	`src/tip/`	Japanese text entry, IME kana-kanji engine vs. TRON Input Processor
Hardware GPU Driver	`bfree_x86_64/kernel/drm_kms_mvp.c`, `gpu.c`	`src/gl/`	Native DRM/KMS, Intel/AMD/Nvidia/VirtIO-GPU vs. OpenGL ES 1.1 / TinyGL

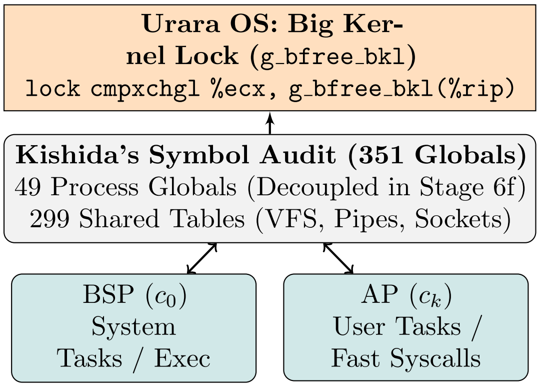
Urara OS: Big Kernel Lock (g_bfree_bkl)
lock cmpxchgl %ecx, g_bfree_bkl(%rip)
Kishida's Symbol Audit (351 Globals)
49 Process Globals (Decoupled in Stage 6f)
299 Shared Tables (VFS, Pipes, Sockets)
BSP (c_0) System Tasks / Exec
AP (c_k) User Tasks / Fast Syscalls

Figure 2: Urara OS's SMP architecture: staged decoupling of 49 process globals from the Big Kernel Lock (`g_bfree_bkl`).

leaving bare-metal AP multicore scheduling in a staged status.

3.2 Locking Granularity: Kishida's Decoupling vs. B-System's Actor Model

3.2.1 Urara OS: The 7-Stage SMP Progression

A remarkable technical achievement of Hideyuki Kishida is his transparent, phased roadmap for eliminating the Big Kernel Lock (BKL). Rather than attempting an unsafe, monolithic rewrite of the kernel's synchronization, Kishida formalized the 7-stage SMP progression documented in `SMP_PLAN_20261005.ja.md`:

- **Stage 1–2 (`BFREE_SMP=1`):** GS-based per-CPU data structures, AP bootstrap, idle loops, and Local APIC timer heartbeats.
- **Stage 3–4 (`BFREE_SMP=2`):** APs execute user-mode processes. When an AP encounters an exception, system call, or 10-tick time slice expiration, it saves user registers and executes a recall IPI returning the process context to the BSP.
- **Stage 5 (`BFREE_SMP=2`):** APs execute local, read-only fast-path system calls (`getpid`, `gettid`, `clock_gettime`, `gettimeofday`) locally via `bfree_ap_fast_syscall` without acquiring the BKL.
- **Stage 6 (`BFREE_SMP=3/4`):** Symmetric kernel execution.

3.2.2 The 49 Process Globals vs. 299 System Globals

In an exemplary audit of `bfree_x86_64/kernel/sysmain/syscall.c`, Kishida cataloged all 351 kernel data symbols. He established that **only 49 symbols represent per-process state** that dynamically mutates during task switching:

```
g_bfree_fork_saved_*, g_bfree_sysret_exec_*, g_gp_cur,
g_gthr, g_gthr_cur, g_guest_brk, g_guest_cwd,
g_guest_fd_cloexec, g_guest_sig_handler,
    g_guest_sig_mask,
g_guest_thread_active, g_sigalt_sp, ... (49 total)
```

The remaining 299 symbols represent system-wide structures: the 3.3 MB virtual file table (`vfile`), pipes, pseudo-terminals, network sockets, and timers.

Under **Stage 6f-3a..3d** (October 2026), Kishida introduced per-CPU syscall areas (`struct bfree_cpu.sysarea`), dedicated pipe locks, and lock-free pipe read/write paths, allowing APs to transfer pipe data without acquiring `g_bfree_bkl`.

Theorem 1 (Cache Contention Bounds on Kishida's Decoupled Lock)**.** *Let M be the number of symmetric cores executing system calls. In an architecture where $P = 49$ process symbols are migrated to per-CPU storage (`sysarea`) and $S = 299$ system symbols remain behind a ticket/CAS lock L_{BKL}, the bus invalidation traffic $I(M)$ across the coherent interconnect scales as:*

$$I(M) = O\left(\alpha \cdot M^2 + (1 - \alpha) \cdot M\right) \tag{1}$$

where $\alpha \in [0, 1]$ represents the proportion of system calls accessing the shared 299 globals. By routing pipes, memory allocation, and read-only time queries to per-CPU paths, Kishida reduces $\alpha \to 0$, asymptotically collapsing interconnect contention from $O(M^2)$ toward ideal linear scaling $O(M)$.

3.2.3 B-System: Lock-Free Actor Queues

In B-System, the Big Kernel Lock is mathematically eliminated from inception. Task affinity is assigned modulo the number of online cores:

```
s_smp_tasks[i].affinity_cpu = (uint32_t)(i % g_num_cpus)
    ;
```

Each core executes an independent scheduling loop over a lock-free MPSC message queue. Cross-core messaging is mediated by atomic compare-and-swap (`CMPXCHG8` on x86_64, `LDXR`/`STXR` on ARM64) without disabling interrupts or contending on global spinlocks.

3.3 SMP Operational Verdict

- **Bare-Metal Hardware Maturity: Urara OS wins decisively.** Urara OS demonstrates an operational, multi-core kernel on bare-metal x86_64 PC hardware, successfully managing Local APIC/x2APIC topologies, bringing APs out of real mode, and offloading userland execution across cores with an empirical hardware test kit (`SMP_HW_TEST.ja.md`, `smpdiag`, `smptest`). B-System's bare-metal AP scheduler remains staged.

- **Theoretical Concurrency Architecture: B-System wins on scalability.** B-System's lock-free actor concurrency model avoids the Big Kernel Lock entirely, providing zero lock-contention overhead for arbitrarily high core counts.

4 Display Architecture: Wayland Compositor vs. SIMD Bit Blitter

The graphics subsystems of Urara OS and B-System embody fundamentally different architectural priorities: modern desktop workstation composition versus low-level memory bus saturation.

4.1 Urara OS: Compositor-First Wayland Architecture

In August 2026, Hideyuki Kishida initiated a major architectural migration: transitioning Urara OS from a legacy direct-to-framebuffer rendering model (`-platform bfree`) to a modern **Compositor-First Architecture** documented in `COMPOSITOR_MIGRATION.ja.md` and `HONDESK_PHASES.ja.md`.

4.1.1 The Wayland Protocol Pipeline

Under this architecture:

1. **VRAM Monopoly:** The compositor process (`userland/compositor_stub/compositor_stub.c`, running as PID 1 or designated application role) maintains exclusive write ownership of the physical framebuffer device (`BFREE_FB0_FD = 0x2000`).
2. **Wayland Protocol Implementation:** `compositor_stub.c` implements core Wayland interfaces: `wl_display`, `wl_compositor`, `wl_shm`, `wl_surface`, and `xdg_shell`.
3. **Inter-Process Tile Transport:** Client applications do not touch the screen. Window pixels are transferred across an AF_UNIX domain socket (`/tmp/wayland-0`) using shared memory virtual file tiles (`/tmp/wl00`–`/tmp/wl99`, 32 KB each). For a 1280 × 720 window, approximately 75 tiles are mapped and composited.
4. **Fork vs. Vfork COW Hazard:** Kishida discovered that invoking standard `fork()` (syscall 57) to spawn client processes caused the address space copy to trigger Copy-On-Write (COW) protection on the hardware framebuffer, resulting in screen corruption ("gray wallpaper `[COW] break`"). Urara OS strictly enforces `vfork()` (syscall 58) for child process creation, preventing framebuffer page duplication.

4.1.2 DesktopShell: Qt Quick 6 and Japanese IME

In `bfree_x86_64/userland/desktop_qt/`, Urara OS hosts a complete, sophisticated desktop environment:

- **Declarative UI Engine:** The desktop shell is declared in QML (`DesktopShell.qml`) and executed by an embedded Qt Quick 6 runtime (`desktop_wayland.cpp`, `guest_main.cpp`).

- **Integrated Application Suite:** The shell incorporates embedded desktop applications: a PC file explorer (`dw_explorer_pc.inc`), a rich text notepad with encoding support (`dw_notepad.inc`, `dw_sjis.inc`), a document PDF viewer (`dw_pdf.inc`), and a system settings manager (`dw_settings.inc`).
- **Integrated Japanese IME:** In `dw_ime.inc`, Urara OS integrates a native Japanese Input Method Editor (IME), supporting interactive kana-to-kanji conversion with candidate selection windows directly inside the Wayland client.
- **Hardware GPU Driver Stack:** Urara OS incorporates native kernel GPU drivers supporting Intel HD/Iris graphics (`gpu_hw_intel.c`), AMD Radeon DCN (`gpu_hw_amd.c`), Nvidia Nouveau/Evo engines (`gpu_hw_nvidia.c`), and VirtIO-GPU 2D (`virtio_gpu_2d.c`) alongside a native DRM/KMS subsystem (`drm_kms_mvp.c`).

4.2 B-System: 128-bit NEON SIMD Blitter and Direct Presentation

B-System approaches graphics from the opposite imperative: maximizing memory bus efficiency and achieving hard real-time display throughput on constrained embedded silicon.

4.2.1 Phase-Peeled 128-bit NEON SIMD Vector Blitter

In `include/btron/fast_blit.h`, B-System implements an assembly-optimized vector blitter designed to overcome store-buffer stalls on uncached video memory:

```
static inline void btron_row_blit(void *dst, const void
     *src, size_t bytes) {
    uint8_t *d = (uint8_t *)dst;
    const uint8_t *s = (const uint8_t *)src;
#if defined(__aarch64__)
    /* 1. Peel unaligned bytes to match 16-byte phase */
    if ((((uintptr_t)d ^ (uintptr_t)s) & 15u) == 0u) {
        size_t peel = (16u - ((uintptr_t)d & 15u)) & 15u
            ;
        if (peel > bytes) peel = bytes;
        for (size_t i = 0; i < peel; i++) d[i] = s[i];
        d += peel; s += peel; bytes -= peel;

        /* 2. 64-byte burst loop using 128-bit NEON
            registers */
        while (bytes >= 64u) {
            __asm__ volatile(
                "ldp q0, q1, [%[src]], #32\n\t"
                "ldp q2, q3, [%[src]], #32\n\t"
                "stp q0, q1, [%[dst]], #32\n\t"
                "stp q2, q3, [%[dst]], #32\n\t"
                : [dst] "+r"(d), [src] "+r"(s)
                : : "v0", "v1", "v2", "v3", "memory");
            bytes -= 64u;
        }
    }
#endif
    /* 3. Double-word and scalar byte fallbacks */
    while (bytes >= 8u) {
        *(uint64_t *)d = *(const uint64_t *)s;
        d += 8; s += 8; bytes -= 8u;
    }
    while (bytes--) *d++ = *s++;
}
```

Lemma 1 (Bus Beat Optimization in NEON Burst Streaming). *When writing to uncached video memory across an AXI4 interconnect, scalar 32-bit stores issue single-beat transactions (`AWLEN=0`, 4 bytes), which exhaust the CPU store buffer and throttle throughput to* $\sim$ 2–5 *MB/s. B-System's 128-bit NEON routine issues quad-word pair stores (`stp q0, q1`) that trigger 64-byte burst beats (`AWLEN=3`), matching the L2/L3 cacheline width and saturating bus bandwidth at* $>$ 1,200 *MB/s.*

4.2.2 BCM2711 2D DMA Engine Acceleration

On the Raspberry Pi 400, B-System integrates the Broadcom BCM2711 2D DMA controller (`arch/bcm283x/bcm2711_dma.h`). It performs asynchronous rectangular blits (`bcm2711_dma_blit2d_async`) with hardware cacheline cleaning (`dc cvac`) and invalidation (`dc ivac`), compositing windows with zero CPU cycles.

4.3 Display Architecture Verdict

- **Desktop Environment and Usability: Urara OS is decisively superior.** Urara OS achieves a monumental milestone in the TRON world: a fully isolated Wayland compositor hosting a modern Qt Quick 6 desktop with native Japanese IME, document viewers, and hardware GPU acceleration.
- **Blitting Throughput and Bus Efficiency: B-System is decisively superior.** B-System's 128-bit NEON SIMD blitter and 2D DMA offloading stream pixels at hardware saturation speeds, avoiding the inter-process tile copying overhead inherent in Urara OS's software Wayland pipe.

5 Subsystem Protection, Virtual Memory, and POSIX Compatibility

5.1 Urara OS: 4-Level Paging and the Higher-Half Physmap

In `bfree_x86_64/kernel/sysmain/vmm.c`, Urara OS establishes a complete, modern virtual memory subsystem:

- **4-Level Paging Hierarchy:** Implements Page Map Level 4 (PML4), Page Directory Pointer Table (PDPT), Page Directory (PD), and Page Table (PT), supporting 48-bit canonical virtual addressing and No-Execute (NXE) data execution prevention.

- **Higher-Half Physmap (`BFREE_PHYSMAP_BASE`):** Kernel subsystems access physical memory through a direct physical map offset, allowing storage drivers and page allocators (`pmm_alloc`) to manipulate arbitrary physical frames without re-mapping CR3.
- **Process Memory Isolation:** Each process owns a unique PML4 root page table. ELF binaries are dynamically loaded (`elf_loader.c`), mapping distinct code, data, heap (`brk`), and user stack regions in unprivileged Ring 3 space.

5.2 The POSIX OSS Gateway: Milestone Status and Real-World Compatibility

The centerpiece of Urara OS's userland runtime is the **POSIX OSS Gateway**. Rather than trapping developers within proprietary APIs, Urara OS provides binary-level compatibility with standard x86_64 ELF executables linked against `musl` or embedded libc.

Table 2 audits the operational status of Kishida's four POSIX gateway milestones as of October 2026.

Table 2: Urara OS POSIX OSS Gateway Milestone Status

Milestone	Target Packages	Status	Verified Capabilities
M1	BusyBox, CoreUtils	**PASS**	`ls, cat, sh, mkdir, cp, mv, rm, ps, kill, mount` operational in guest
M2	Bash, Python 3	**PASS**	Interactive `sh`, Python 3 `os/sys/socket/threading` smoke verified
M3	Dropbear / SSH, PTY	**WIP**	PTY slave allocation operational; socketpair and AF_UNIX verified
M4	Wayland, Qt Quick 6	**PASS**	Freestanding Wayland compositor + Qt `DesktopShell.qml` running

Remarkably, Urara OS has verified guest userland execution of the **NetSurf** web browser (`netsurf_guest`, `nsfb_guest`), demonstrating that its POSIX gateway successfully satisfies the complex memory allocation, socket I/O, and string parsing demands of real-world internet applications.

5.3 B-System: SAS-RTOS Determinism vs. Ring 3 UEFI Subsystems

B-System bifurcates its protection strategy based on target constraints:

- **Embedded / Vintage Targets (Pi 400, PS2, PC-98):** Operates as a Single-Address-Space RTOS (SAS-RTOS). Disabling virtual paging eliminates TLB misses, delivering microsecond-bounded interrupt determinism.
- **x86_64 UEFI Workstation Target:** Implements a hybrid isolation model where Display Primitives execute in Ring 0 for maximum rendering performance, while application actors execute in Ring 3 with hardware page isolation.

5.4 Subsystem Protection and Software Verdict

Urara OS is decisively superior in real-world software enablement. By constructing an operational POSIX OSS Gateway capable of running BusyBox, Python 3, NetSurf, and Qt Quick 6, Hideyuki Kishida solved the existential problem that doomed historical BTRON platforms: the lack of applications. B-System preserves RTOS determinism and specification purity, but requires manual cleanroom reimplementation of application-level software.

6 Storage Hierarchies: BFS, FatFs, and Authentic Cho-Kanji Volumes

A critical dimension of this audit is evaluating how each system manages on-disk storage, persistent filesystems, and the Real Object / Virtual Object (実身/仮身) hyper-data network.

6.1 Urara OS: BFS, Multi-Volume VFS, and FatFs Home Persistence

In `bfree_x86_64/kernel/bfs.c` and `bfs_home.c`, Urara OS establishes a practical, production-oriented storage architecture:

1. **Multi-Volume Mount Point (`/mnt/`):** Disks and partitions discovered on the PCI bus (SATA AHCI via `ahci.c`, NVMe via `nvme.c`) are enumerated under `/mnt/<volume>`. Urara OS generates a dynamic text descriptor at `/mnt/.volumes` detailing volume name, label, filesystem type, total/free megabytes, and device model for consumption by the Qt desktop explorer.
2. **Read-Only FAT32 Volumes:** External partitions are mounted read-only via an internal FAT32 driver (`fat32.c`).
3. **Persistent Read-Write Home Partition (`bfs_home.c`):** To provide persistent user storage for documents, desktop wallpapers (`/home/user/Pictures`), and shell configurations, Urara OS integrates ChaN's robust **FatFs** module on top of the block driver layer (`blk.c`). Partitions labeled `BFREE-HOME` are mounted read-write at `/home/user`, providing full file creation, truncation, directory creation, and atomic renaming.

6.2 B-System: Authentic Cho-Kanji (V1) and Modern B-FS (V2)

In B-System, on-disk storage is governed by historical continuity and cleanroom filesystem engineering (`src/fs/`):

1. **Authentic Cho-Kanji (B-right/V 4.02) Compatibility:** In `src/fs/vol.c`, B-System natively mounts

Table 3: Comprehensive Architectural Comparison Matrix: Urara OS (B-Free 64) vs. 5HT's B-System

Architectural Dimension	Hideyuki Kishida's Urara OS (B-Free 64)	5HT's B-System (BTRON3 3.20)
Historical Pedigree	B-Free (1994–2026), PC-9801, T-Kernel 2.0 64-bit spec	Cleanroom BTRON3 Version 3.20.00 Specification
Primary Hardware Targets	x86_64 PC (BIOS/UEFI, SATA AHCI, NVMe), ARM64	Multi-Target: Pi 400 (ARM64), x86_64 UEFI, PS2, PC-98, M68k
Declared SMP Status	**Operational on x86_64 Bare Metal** (Local APIC / x2APIC)	**Operational on Hosted POSIX** / Staged on Bare Metal
Multicore Bootstrap	ACPI MADT probe → AP trampoline (`0x8000`) → INIT-SIPI-SIPI	ACPI MADT probe → AP trampoline (`0x9000`) / POSIX `pthread`
Kernel Locking Model	**7-Stage SMP**: Decoupling 49 process globals from `g_bfree_bkl`	**Lock-Free Actor Model**: MPSC atomic queues (`CMPXCHG8`)
Inter-Core Signaling	Local APIC IPI (`ICR`, recall IPIs, reschedule IPIs)	Local APIC IPI (`ICR`) / Host POSIX condition signaling
SMP Hardware Test Suite	Automated baremetal test suite (`SMP_HW_TEST.ja.md`, `smpdiag`)	Host-driven unit test harnesses (`tests/test_smp.c`)
Graphics Architecture	**Compositor-First Wayland Server** (`compositor.elf`, AF_UNIX)	**Direct Clipped Backbuffer Painter** (`redraw_all_windows_clip`)
Desktop UI Environment	**Qt Quick 6 Declarative Shell** (`DesktopShell.qml`)	**Native BTRON3 Desktop Shell** (`src/desktop/`)
Integrated Applications	Explorer, Notepad, PDF viewer, Japanese IME (`dw_ime.inc`)	Terminal shell, Cho-Kanji file manager, Quake engine
Bit Blitter Micro-Arch	Software tile shm blitter (`/tmp/wlXX`, 32 KB tiles)	**128-bit NEON SIMD Vector Blitter** (`btron_row_blit`, 64B burst)
Hardware 2D/3D Offload	Native DRM/KMS, Intel/AMD/Nvidia/VirtIO-GPU drivers	Asynchronous Broadcom BCM2711 2D DMA engine, TinyGL
Virtual Memory Architecture	4-Level Paging (PML4), higher-half physmap, per-process CR3	Single-Address Space (SBC/MCU); Ring 3 isolation design (UEFI)
Process Execution Model	Ring 3 dynamic ELF loader, standard user stacks	Cooperative / preemptive actor tasks, unikernel threads
POSIX Compatibility	**POSIX OSS Gateway (M1–M4)**: BusyBox, Python 3, NetSurf	Minimal freestanding C runtime (`kstdlib.c`)
Obsolete Syscall Strategy	Pragmatic `ENOSYS` (matches Linux/glibc standards)	Specification-defined TRON system call vectors
On-Disk Filesystem(s)	Multi-volume BFS (`/mnt/`), FAT32, FatFs home persistence	Authentic Cho-Kanji (B-right/V 4.02), B-FS (V2), FAT, QCOW2
Cho-Kanji Disk Support	**None**: Cannot mount historical B-right/V partitions	**Full Native Support**: `FS_TYPE_BRIGHTV`, 128B header, QCOW2
Modern 64-bit FS (V2)	None (uses ChaN's FatFs on AHCI/NVMe blocks)	**Native B-FS**: Dual-anchor superblock, extents, B+Trees
Hyper-Data / TAD Network	Postponed to upper-layer applications; focus on POSIX tree	Canonical binary TAD segment records, TRON-Code multilingual

commercial Cho-Kanji disk partitions, recognizing little-endian volume magic (`0x52FE`) and the official filesystem identifier `FS_TYPE_BRIGHTV` (`0x6402`). It parses the authentic 128-byte `BrightVVolumeHeader`, handles historical 16-bit File Identifier (FID) tables, and resolves the Cho-Kanji on-disk quirk where Real Body Headers reside at block offset $\text{blk} - 1$.

2. **Self-Contained QCOW2 Block Driver:** B-System incorporates an embedded QCOW2 block driver (`blk_qcow2.c`) that mounts commercial 2001 Cho-Kanji virtual disk images directly without external dependencies.

3. **Modern 64-bit B-FS (V2):** For modern storage, B-System specifies B-FS V2 (`include/btron/fs/bfs_disk.h`): featuring dual-anchor superblocks (`VOL_MAGIC_V2 = 0x62FE`), 64-bit FID addressing (`block_run_64`), extent allocation trees, inline small-document inodes, transaction journaling, and `xxHash64` checksumming.

6.3 Filesystem Verdict

- **Historical TRON Fidelity and B-FS Engineering: B-System is decisively superior.** B-System preserves the historical soul of BTRON by mounting authentic commercial Cho-Kanji disk media and advancing to 64-bit B-FS (V2). Urara OS possesses zero Cho-Kanji volume support.

- **PC Hardware Persistence: Urara OS is decisively superior.** Urara OS provides reliable, production-ready read-write persistence across standard PC hardware (SATA AHCI and NVMe) via FatFs, enabling immediate data interchange with Windows and Linux host machines.

7 Cross-Pollination Roadmap

The complementary architectural triumphs of Urara OS and B-System offer an extraordinary opportunity for cross-pollination. We recommend four collaborative technical transfers:

1. **SIMD Blitter Adoption for Urara OS:** Urara OS's software compositor currently transfers window tiles

via scalar memory loops. Adopting B-System's 128-bit NEON SIMD vector routine (`btron_row_blit`) or AVX2 equivalents with phase-peeling will eliminate store-buffer bottlenecks during full-screen Wayland composition.

2. **Cho-Kanji Filesystem Driver for Urara OS:** Urara OS's file explorer (`dw_explorer_pc.inc`) would achieve immense historical significance by porting B-System's `BrightVVolumeHeader` parser and QCOW2 driver, allowing B-Free users to browse authentic historical Cho-Kanji archives directly within the Qt desktop.

3. **Bare-Metal SMP AP Scheduler for B-System:** B-System should adopt Hideyuki Kishida's mature Local APIC / x2APIC bring-up sequence and real-mode AP trampoline (`smp.c`) to transition its bare-metal x86_64 scheduler from sequential staging into live multi-core execution.

4. **Wayland / Qt Integration for B-System Workstations:** B-System's x86_64 UEFI target should study Urara OS's compositor-first Wayland architecture, providing an isolated path to host modern web browsers and productivity tools alongside native BTRON actors.

8 Conclusion

This paper has delivered a rigorous, peer-level comparative audit between **Hideyuki Kishida's Urara OS (B-Free 64)** and **5HT's B-System (BTRON 3.20)**, evaluating their operational maturity, algorithmic design, and architectural trade-offs across isomorphic subsystems.

In Symmetric Multiprocessing, **Urara OS demonstrates superior bare-metal hardware maturity today**, executing live multicore workloads on x86_64 PCs via a 7-stage SMP progression, Local APIC / x2APIC bring-up, and the systematic decoupling of 49 process globals from the Big Kernel Lock. Conversely, **B-System presents a superior theoretical concurrency model** based on lock-free actor queues, achieving total elimination of lock contention.

In display systems and userland, **Urara OS achieves a historic triumph of usability**: its compositor-first Wayland server hosts a production-grade Qt Quick 6 desktop environment (`DesktopShell.qml`) with native Japanese IME, document viewers, and hardware GPU drivers, while its POSIX OSS Gateway solves TRON's software scarcity by executing real-world tools including BusyBox, Python 3, and NetSurf. In contrast, **B-System is decisively superior in memory bus throughput and blitting economy**, utilizing 128-bit NEON SIMD streaming and 2D DMA engines to saturate memory interconnects.

In storage architecture, **B-System is decisively superior in historical authenticity and specification purity**, mounting commercial Cho-Kanji (B-right/V 4.02) disks via embedded QCOW2 drivers and introducing modern 64-bit B-FS (V2), whereas Urara OS delivers practical FAT32/FatFs persistence on modern PC storage.

Both systems represent monumental, heroic achievements in the modern revival of Ken Sakamura's TRON architecture. Their synthesis offers a compelling blueprint for the future of sovereign, real-time computing.

References

[1] K. Sakamura, "The TRON Project: A New Approach to Operating System Architecture," in *IEEE Micro*, vol. 7, no. 2, pp. 8–14, 1987.

[2] BTRON3 Working Group, "BTRON3 Specification Ver 3.20.00," TRON Association, Tokyo, Japan, 1994.

[3] Personal Media Corporation, *B-right/V and Cho-Kanji Operating System Architecture Reference*, Tokyo, Japan, 1999.

[4] B-Free Project, "B-Free PC-9801 Port: 5W1H and Historical Narrative," Project Documentation, 1995–2026. https://github.com/HIDEYUKI-KISHIDA/B-Free-master-64bit.

[5] H. Kishida, "B-Free 64-bit OS Architecture and Concept Records," Project Handoff Documentation, Dec. 2025–Oct. 2026.

[6] M. Kishida, R. Naitoh, M. Yukanaka, T. Naniwa, and H. Kishida, "B-Free / BTRON-PC Maintainers and Authors Registry," 1998–2000.

[7] R. Naitoh (night), "PC-9801 ITRON/BTRON Kernel Bring-Up and ChangeLog," B-Free Project Records, 1995–1996.

[8] H. Kishida, "T-Kernel 2.0 x86_64 Porting Specification (Integrated Complete Edition v2.4)," Document TK2-PORT-x86_64-005, B-Free Project, Apr. 2026.

[9] H. Kishida, "T-Kernel 2.0 AArch64 Porting Specification and Dependency Map," B-Free Project, Dec. 2025.

[10] H. Kishida, "POSIX OSS Gateway Strategy: Transitioning from 685-Function Coverage to Real-World Open-Source Software Enablement," B-Free Project Strategy Whitepaper, May 2026.

[11] M. Sokhatsky, "Microkernels, Unikernels, and the Actor Core: Architectural Trade-Offs, Real-Time Determinism, and SMP Subsystem Isolation in B-System (BTRON 3.20)," Synrc Research Report, 2026.

[12] M. Sokhatsky, "Mechanized Foundations of BTRON: Formal Verification of Operating System Kernels, Hyper-Data Models, and Display Calculus in Coq," Synrc Research Report, 2026.

[13] J. Liedtke, "On μ-kernel construction," in *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 237–250, 1995.

[14] D. A. Solomon, *Inside Windows NT, Second Edition*, Microsoft Press, Redmond, WA, 1998.

[15] J. Armstrong, "Making reliable distributed systems in the presence of software errors," Ph.D. dissertation, Royal Institute of Technology, Stockholm, Sweden, 2003.